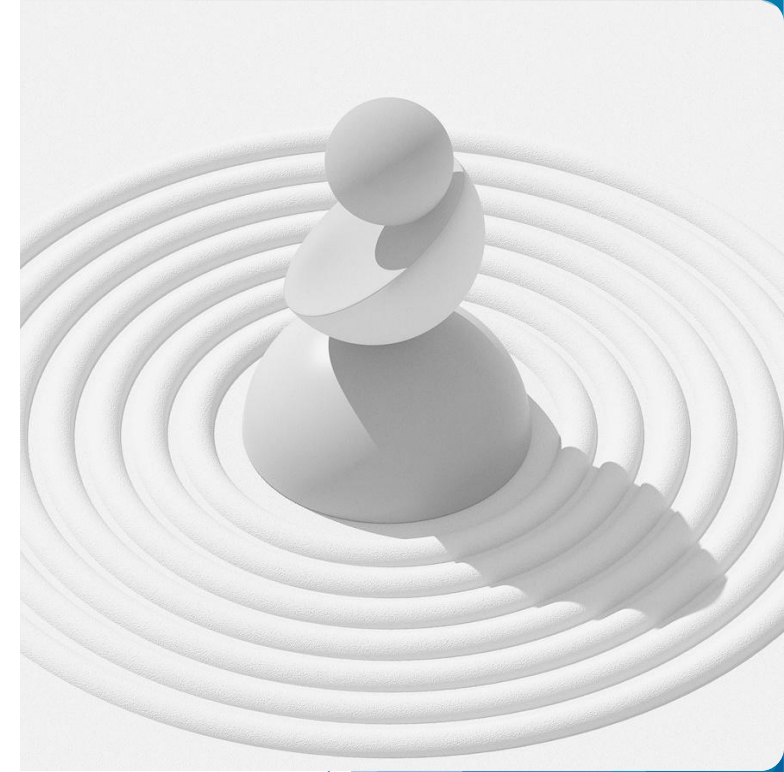


Presented by Nirvana Popescu

Lesson 5: Software Design Principles in Safety-Critical Software Engineering and Best Practices

Outline

- ▶ Objectives of Software Design Principles
- ▶ Why Design Matters More in Safety-Critical Systems
- ▶ Safety Standards Drive the Software Design
- ▶ Core Design Principles for Safety-Critical Software
- ▶ Top-Down Design
- ▶ Modularity and Information Hiding
- ▶ Traceability by Design
- ▶ Strong Typing and Deterministic Design
- ▶ SOLID Principles
- ▶ DRY and KISS
- ▶ Model-Based Design
- ▶ Best Practices for Safety-Critical Design



Software Design Principles and Best Practices for Safety-Critical Systems

► Objectives

- Understand why software design is different in safety-critical domains
- Learn the fundamental design principles used in certified systems
- Explore how standards influence software architecture
- See how Model-Based Design supports compliance

► Domains

- Aerospace
- Automotive
- Railway
- Medical devices
- Industrial automation



Why Design Matters More in Safety-Critical Systems

► General Software vs Safety-Critical Software

General Software	Safety-Critical Software
Optimized for features	Optimized for safety
Flexible architecture	Deterministic architecture
Bugs can be fixed after release	Failures shall be prevented
Testing is important	Verification required by standards
Documentation optional	Documentation mandatory

► Design Goals

- Predictability
- Reliability
- Maintainability
- Verifiability
- Certification readiness

Safety Standards Drive the Software Design

► Major Standards:

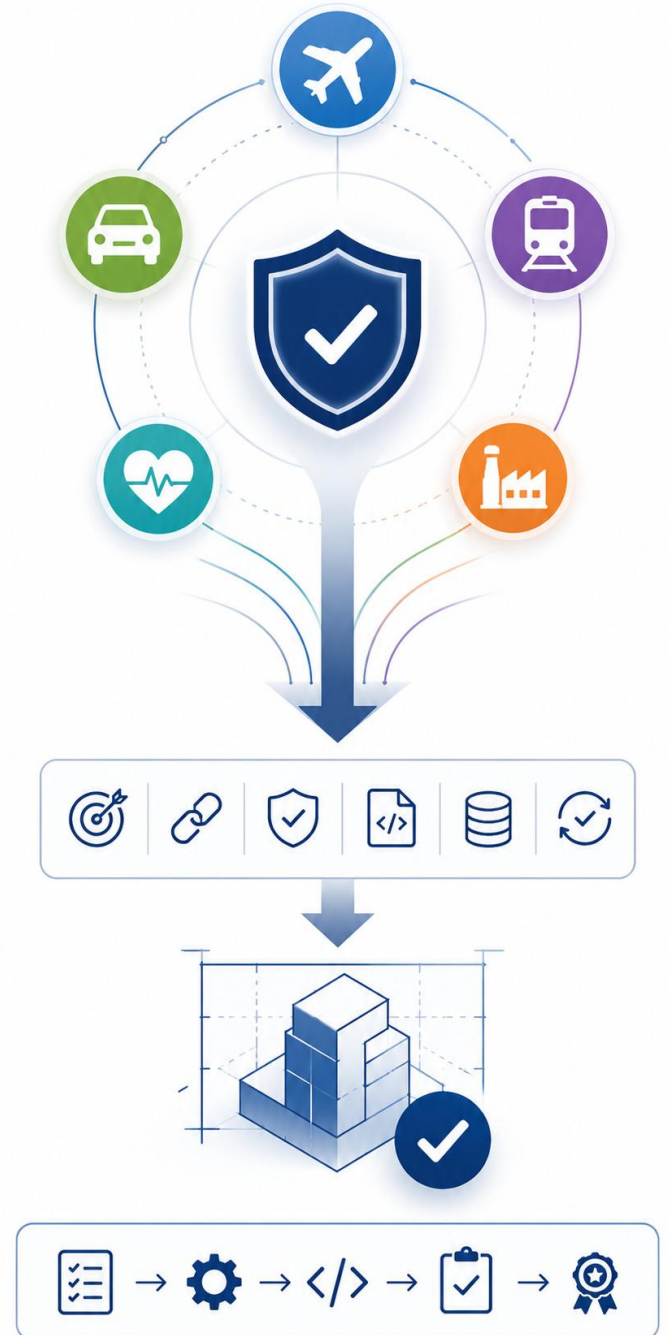
Domain	Standard
Aerospace	DO-178C
Automotive	ISO 26262
Railway	EN 50716
Medical Devices	IEC 62304
Industrial	IEC 61508

► Standards Require

- Requirements-based design
- Traceability
- Independent verification
- Coding guidelines
- Deterministic behavior
- Configuration management

► Design Implication

- The architecture must support certification from the beginning—not as an afterthought.



Core Design Principles for Safety-Critical Software

Principle	Purpose in Safety-Critical Systems
Top-Down Design	Decompose complex systems into manageable hierarchical components
Modularity	Isolate functionality to improve maintainability and fault containment
Traceability	Link requirements, design, implementation, and verification artifacts
Strong (Hard) Typing	Prevent type-related errors and improve software robustness
SOLID Principles	Improve maintainability, extensibility, and testability
DRY (Don't Repeat Yourself)	Eliminate duplicated logic to reduce maintenance errors
KISS (Keep It Simple)	Favor simple, understandable solutions that are easier to verify
Model-Based Design	Develop executable models that support simulation, verification, and certified code generation

- ▶ All these principles collectively support the objectives required by standards such as DO-178C, ISO 26262, EN 50716, IEC 62304, and IEC 61508.



SUPPORTED BY INTERNATIONAL SAFETY STANDARDS



Top-Down Design

▶ Principle

- ▶ Start from the system objectives and progressively decompose into smaller components.

Mission ➡ System ➡ Subsystems ➡ Components ➡ Functions ➡ Implementation

▶ Benefits

- ▶ Easier verification
- ▶ Clear interfaces
- ▶ Better requirement allocation
- ▶ Reduced complexity
- ▶ Supports certification reviews

▶ In Scade One

- ▶ Hierarchical operators
- ▶ Modular architecture
- ▶ Incremental refinement

Practical Example - Emergency Braking

Mission → Prevent collision

System → Emergency Braking System

Subsystems → Sensing | Decision | Braking

Components → Distance Sensor | Brake
Controller | Actuator

Functions → Detect obstacle | Calculate risk
| Apply brake

Implementation → SCADE operators +
generated embedded code

Modularity and Information Hiding

- ▶ Divide the system into independent functional modules.
- ▶ Examples
 - ▶ Sensor Processing
 - ▶ Diagnostics
 - ▶ Navigation
 - ▶ Brake Controller
 - ▶ Human-Machine Interface
- ▶ **Benefits**
 - ▶ Easier maintenance
 - ▶ Independent verification
 - ▶ Fault isolation
 - ▶ Reusability
 - ▶ Parallel development
- ▶ **Information Hiding**
 - ▶ Modules expose only necessary interfaces while hiding implementation details.
- ▶ **Safety Benefit**
 - ▶ A defect remains localized instead of propagating across the system.



Traceability by Design

▶ Traceability Links

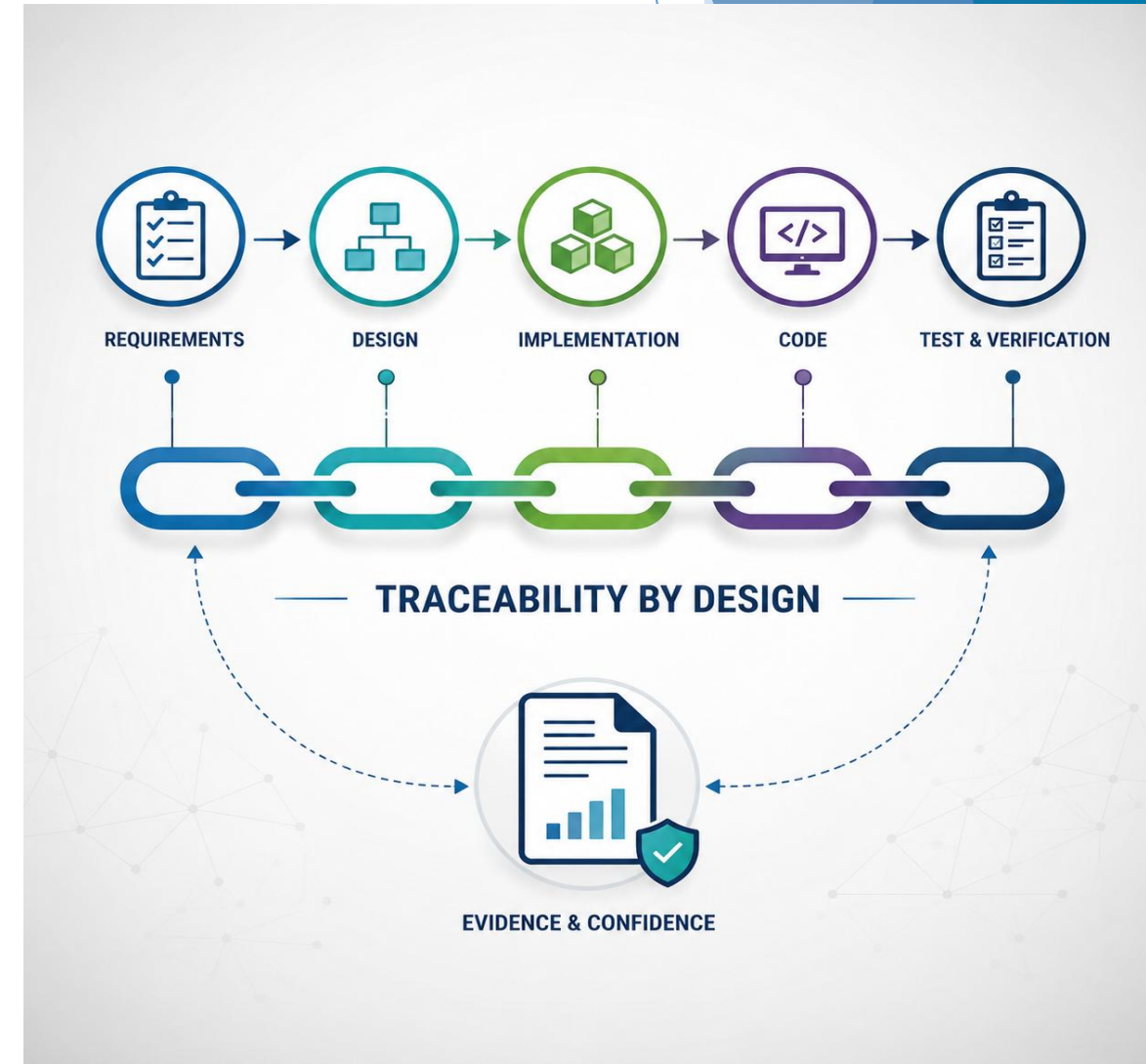
Requirement ➡ Architecture ➡ Model ➡ Generated Code ➡ Test cases

▶ Why It Matters

- ▶ Demonstrates compliance
- ▶ Supports impact analysis
- ▶ Simplifies certification audits
- ▶ Detects missing requirements
- ▶ Enables change management

▶ Scade One

- ▶ Automatic traceability between
 - ▶ Models
 - ▶ Generated code
 - ▶ Verification artifacts



Strong Typing and Deterministic Design

- ▶ **Hard (Strong) Typing**
 - ▶ Variables have explicit data types. ➡
- ▶ **Prevents**
 - ▶ Invalid assignments
 - ▶ Overflow errors ➡
 - ▶ Type conversion mistakes
 - ▶ Runtime surprises
- ▶ **Deterministic Execution**
 - ▶ Safety-critical software must always produce identical outputs for identical inputs.
- ▶ **No:**
 - ▶ Hidden side effects
 - ▶ Undefined behavior
 - ▶ Dynamic interpretation
- ▶ **Swan Language**
 - ▶ Supports strong typing and deterministic execution.

Example:

Altitude : int32
Speed : float32
BrakeCommand : bool

Real-Life Failure: Ariane 5 (1996)

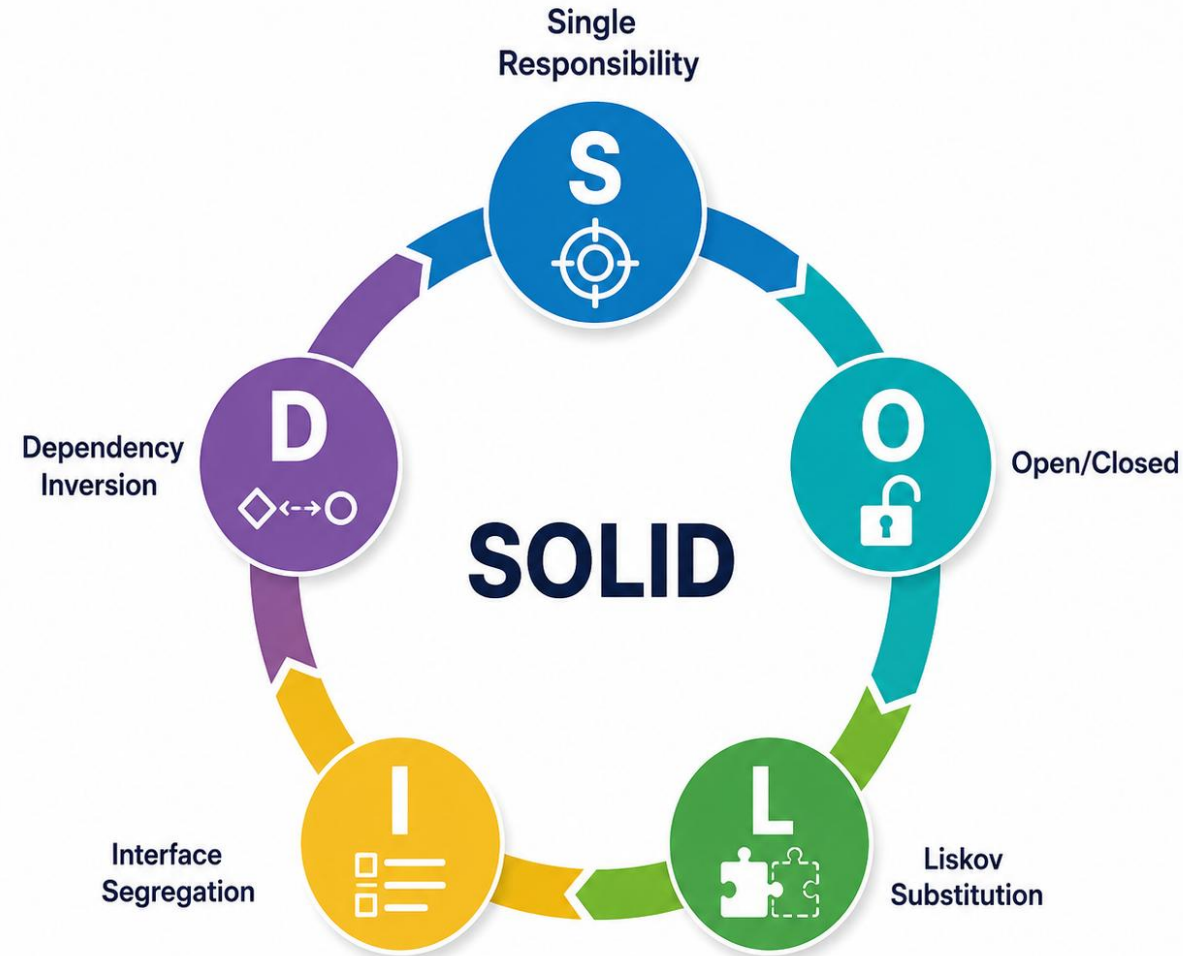
64-bit floating point → 16-bit integer
Value exceeded integer range → Overflow
↓
Inertial reference system failure
↓
Loss of guidance → Rocket destroyed



Unsafe type conversions can cause catastrophic system failures.

SOLID Principles

- ▶ **Single Responsibility**
 - ▶ Each component has one well-defined function.
- ▶ **Open/Closed**
 - ▶ Extend functionality without modifying certified components.
- ▶ **Liskov Substitution**
 - ▶ Replacement should preserve behavior.
- ▶ **Interface Segregation**
 - ▶ Small specialized interfaces reduce coupling.
- ▶ **Dependency Inversion**
 - ▶ Depend on abstractions rather than implementations.
- ▶ **Safety Perspective - SOLID improves:**
 - ▶ Maintainability
 - ▶ Testability
 - ▶ Certification effort



Example: Automotive Brake Controller

- S - One component → one responsibility
- O - Add sensors without changing validated control logic
- L - Sensors can be substituted safely
- I - Small, specialized sensor interfaces
- D - Controller depends on interfaces, not hardware drivers

DRY and KISS

DRY (Don't Repeat Yourself)

- ▶ Avoid duplicated logic.
- ▶ Benefits
 - ▶ Fewer bugs
 - ▶ Easier updates
 - ▶ Consistent behavior

KISS (Keep It Simple)

- ▶ Simple solutions are
 - ▶ Easier to verify
 - ▶ Easier to test
 - ▶ Easier to certify

▶ Safety Rule

- ▶ Simple software is generally safer software.

▶ However

- ▶ Avoid excessive abstraction that hides safety behavior.

Example - Brake Controller

DRY:

- Do not make Obstacle-distance validation repeated in BrakeControl, WarningSystem, and Diagnostics
- Implement **one validated CheckDistance() function** and reuse it.

KISS:

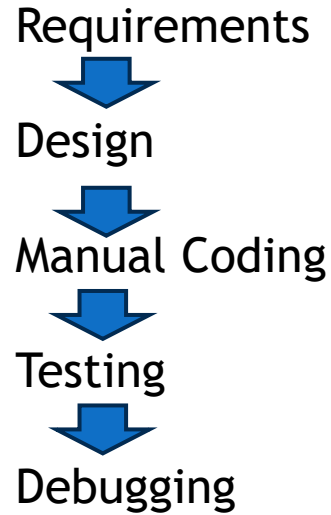
- No complex multi-layer logic for emergency braking
- Take simple, explicit decision:
Obstacle detected → Risk threshold exceeded → Apply brake

DRY asks: "Am I implementing the same logic several times?"

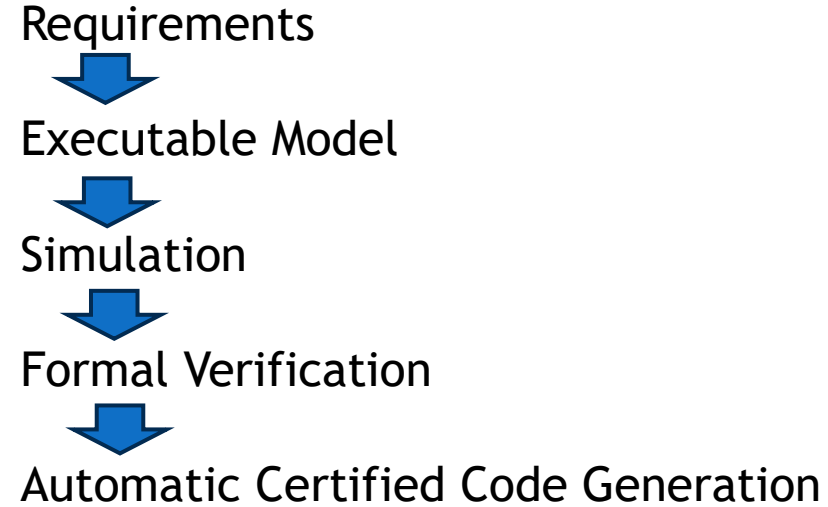
KISS asks: "Am I making the solution more complicated than necessary?"

Model-Based Design with ANSYS Scade One

Traditional Development



Model-Based Development



► Advantages

- Fewer implementation errors
- Earlier validation
- Consistent documentation
- Automatic traceability
- Certified code generation



Best Practices for Safety-Critical Design

▶ Recommended Practices

- ▶ Design for determinism
- ▶ Keep modules cohesive
- ▶ Minimize coupling
- ▶ Define explicit interfaces
- ▶ Use strong typing
- ▶ Eliminate dead code
- ▶ Maintain bidirectional traceability
- ▶ Verify continuously
- ▶ Follow coding standards (e.g., MISRA)
- ▶ Use qualified tools

▶ Common Mistakes

- ▶ Overly complex architecture
- ▶ Hidden dependencies
- ▶ Poor documentation
- ▶ Weak traceability
- ▶ Large monolithic modules

Summary

- ▶ **Safety-critical software requires a different design mindset**
Safety, predictability, and certification take priority over functionality and rapid development.
- ▶ **Software architecture must support certification from the beginning**
Standards such as **DO-178C**, **ISO 26262**, and **EN 50716** influence every design decision.
- ▶ **Good design principles reduce complexity and improve reliability**
Top-Down Design, Modularity, Traceability, Strong Typing, SOLID, DRY, and KISS work together to create maintainable and verifiable software.
- ▶ **Model-Based Design improves quality and reduces certification effort**
Executable models, formal verification, and qualified code generation with **ANSYS Scade One** increase confidence in the final software.
- ▶ **Safe software is the result of disciplined engineering**
Applying best practices consistently throughout the development lifecycle leads to software that is reliable, maintainable, verifiable, and certifiable.

